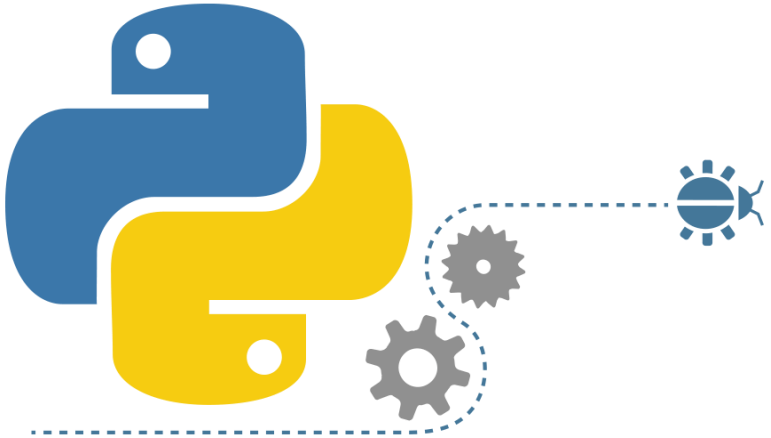
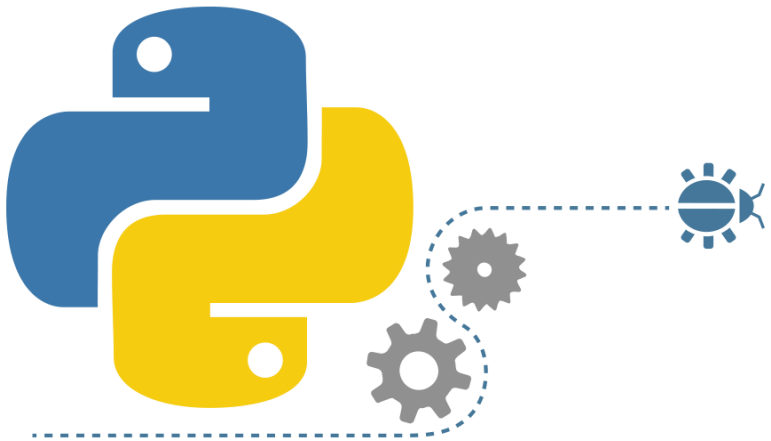
Tuple - Dictionary

1. Định nghĩa Tuple
2. Tuple hơn list ở điểm nào?
3. Tạo một tuple
4. Truy cập vào các phần tử của tuple
5. Thay đổi một tuple
6. Xóa tuple
7. Phương thức và hàm dùng với tuple trong Python
8. Kiểm tra phần tử trong tuple
9. Lặp qua các phần tử của tuple trong Python
10. Định nghĩa Dictionary
11. Cách tạo dictionary trong Python
12. Truy cập phần tử của dictionary
13. Thay đổi, thêm phần tử cho dictionary
14. Xóa phần tử từ dictionary
15. Các phương thức và hàm cho dictionary
16. Dictionary comprehension trong Python
17. Kiểm tra và lặp qua phần tử trong dictionary
18. Bài tập rèn luyện Dictionary



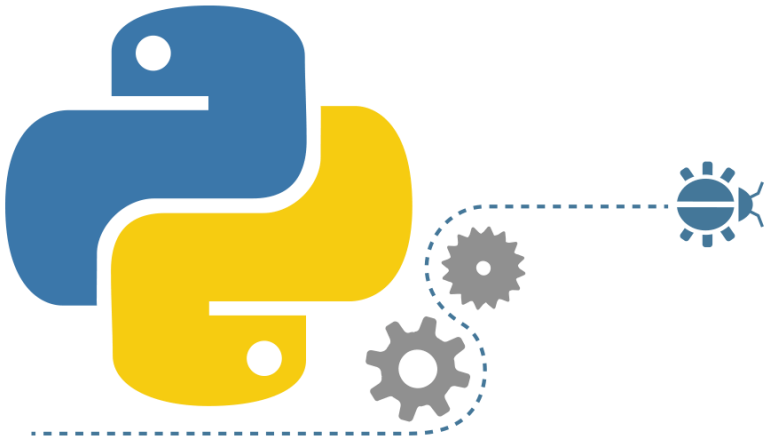
1. Định nghĩa Tuple



Nội dung bài học

Tuple trong Python là một chuỗi các phần tử có thứ tự giống như list. Sự khác biệt giữa list và tuple là chúng ta không thể thay đổi các phần tử trong tuple khi đã gán, nhưng trong list thì các phần tử có thể thay đổi.

Tuple thường được sử dụng cho các dữ liệu không cho phép sửa đổi và nhanh hơn list vì nó không thể thay đổi tự động. Một tuple được định nghĩa bằng dấu ngoặc đơn (), các phần tử trong tuple cách nhau bằng dấu phẩy (,).



Nội dung bài học

Ví dụ:

```
t = (10, "This is my Python", 2j)
```

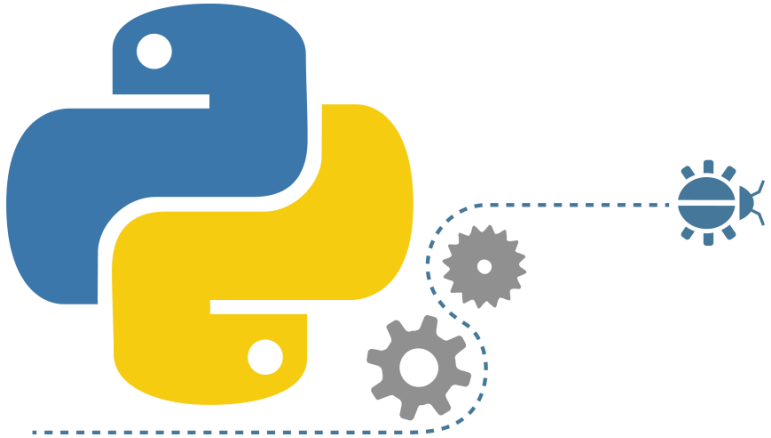
Bạn có thể sử dụng toán tử cắt [] để trích xuất phần tử trong tuple nhưng không thể thay đổi giá trị của nó.

```
t = (10, "This is my Python", 2j)
#t[0:2] = (10, 'This is my Python')
print("t[0:2] = ", t[0:2])
```

Chạy code trên ta được kết quả:

```
t[0:2] = (10, 'This is my Python ')
```

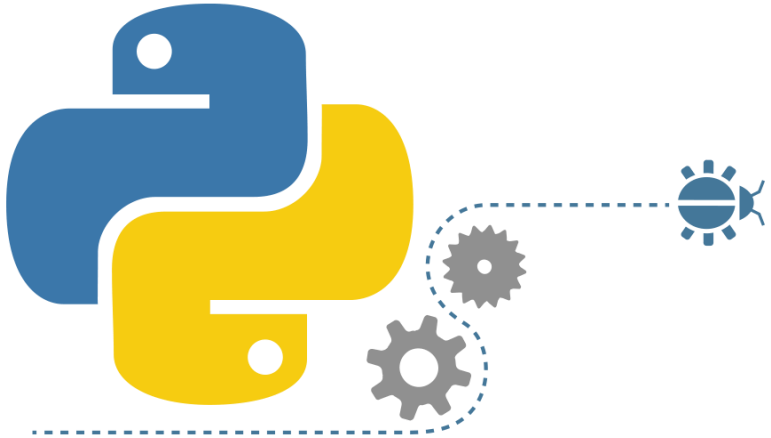
2. Tuple hơn list ở điểm nào?



Nội dung bài học

- Vì tuple và list khá giống nhau, nên chúng thường được sử dụng trong những tình huống tương tự nhau. Tuy nhiên, tuple vẫn có những lợi thế nhất định so với list, như:
- Tuple thường được sử dụng cho các kiểu dữ liệu không đồng nhất (khác nhau) và list thường sử dụng cho các kiểu dữ liệu (đồng nhất) giống nhau.
- Vì tuple không thể thay đổi, việc lặp qua các phần tử của tuple nhanh hơn so với list. Vì vậy, trong trường hợp này tuple chiếm ưu thế về hiệu suất hơn list một chút.
- Tuple chứa những phần tử không thay đổi, có thể được sử dụng như key cho dictionary. Với list, điều này không thể làm được.
- Nếu có dữ liệu không thay đổi việc triển khai nó như một tuple sẽ đảm bảo rằng dữ liệu đó được bảo vệ chống ghi (write-protected).

3. Tạo một tuple



Nội dung bài học

- Tuple được tạo bằng cách đặt tất cả các phần tử của nó trong dấu ngoặc đơn (), phân tách bằng dấu phẩy. Bạn có thể bỏ dấu ngoặc đơn nếu muốn, nhưng nên thêm nó vào cho code rõ ràng hơn.
- Tuple không bị giới hạn số lượng phần tử và có thể có nhiều kiểu dữ liệu khác nhau như số nguyên, số thập phân, list, string,...

```
# tuple số nguyên
my_tuple = (2, 4, 16, 256)
```

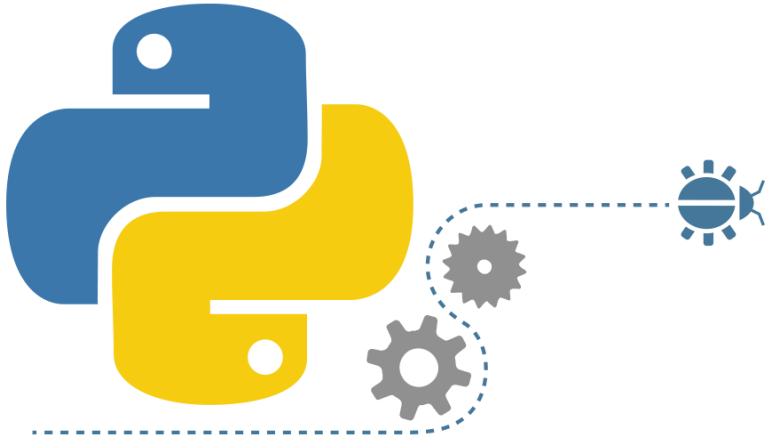
```
# tuple có nhiều kiểu dữ liệu
my_tuple = (10, "This is my Python", 3.5)
```

```
# tuple lồng nhau
my_tuple = ("ABC", [2, 4, 6], (3, 5, 7))
```

```
# tuple có thể được tạo mà không cần dấu ()
my_tuple = 10, "This is my Python", 3.5
```

Tạo tuple **chỉ có một phần tử** hơi phức tạp chút, nếu bạn tạo theo cách thông thường là cho phần tử đó vào trong cặp dấu () là chưa đủ, cần phải thêm dấu phẩy để chỉ ra rằng, đây là tuple.

4. Truy cập vào các phần tử của tuple



Nội dung bài học

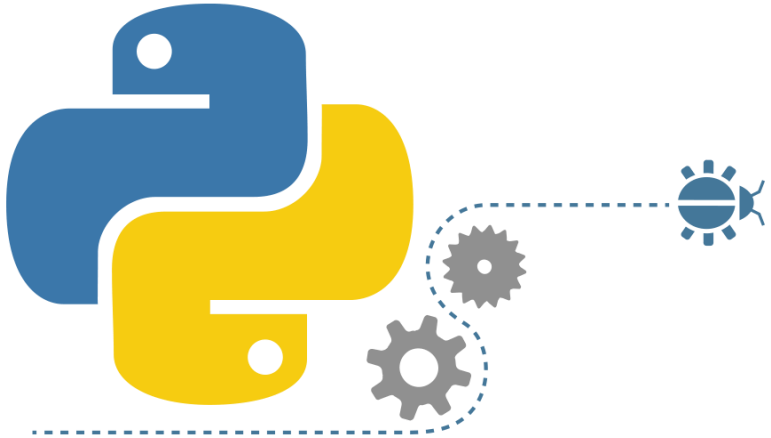
- Giống truy cập phần tử List
- **Index:** Sử dụng toán tử index [], với index bắt đầu bằng 0.
  - tuple có 6 phần tử thì index bắt đầu từ 0 đến 5.
  - Nếu cố gắng truy cập đến index 6, 7 thì sẽ tạo lỗi IndexError.
  - Index bắt buộc phải là số nguyên, mà không thể là số thập phân hay bất kỳ kiểu dữ liệu nào khác, nếu không sẽ tạo lỗi TypeError.
- Những tuple lồng nhau được truy cập bằng cách sử dụng index lồng nhau:

```
# tuple lồng nhau
n_tuple = ("This is my Python", [2, 6, 8], (1, 2, 3))
# index lồng nhau
# Output: 'r'
print(n_tuple[0][5])
# index lồng nhau
# Output: 8 print(n_tuple[1][2])
```

Index âm: Python cho phép lập chỉ mục âm cho các đối tượng dạng chuỗi. Index -1 tham chiếu đến phần tử cuối cùng, -2 là thứ 2 tính từ cuối tính lên.

Cắt lát: Có thể truy cập đến một loạt phần tử trong tuple bằng cách sử dụng toán tử cắt lát : (dấu 2 chấm).

5. Thay đổi một tuple



Nội dung bài học

- tuple không thể thay đổi một khi đã được gán. Nhưng, nếu phần tử là một kiểu dữ liệu có thể thay đổi (như list chẳng hạn) lồng nhau có thể được thay đổi. Chúng ta cũng có thể gán giá trị khác cho tuple (gọi là gán lại - reassignment).

```
my_tuple = (1, 3, 5, [7, 9])
#my_tuple[1] = 9  =>  # TypeError: 'tuple' object does not support item assignment
```

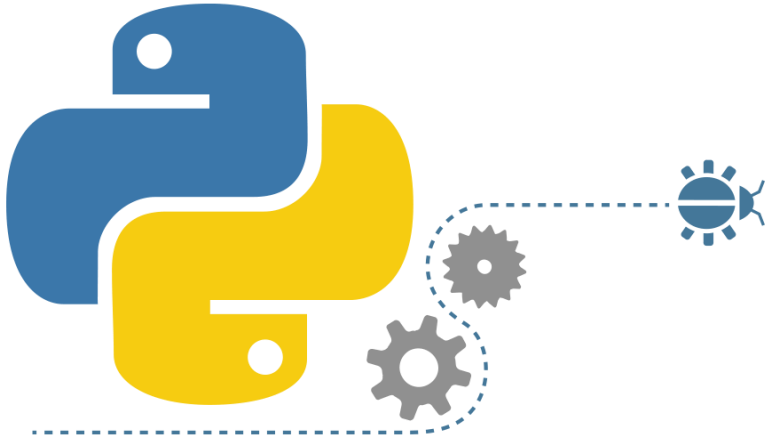
```
my_tuple[3][0] = 8
# Output: (1, 3, 5, [8, 9])
```

```
# Nếu cần thay đổi tuple hãy gán lại giá trị cho nó
# Output: ('q', 'u', 'a', 'n', 't', 'r', 'i', 'm', 'a', 'n', 'g')
my_tuple = ('q', 'u', 'a', 'n', 't', 'r', 'i', 'm', 'a', 'n', 'g')
print(my_tuple)
```

Có thể dùng toán tử + để nối 2 tuple, toán tử * để lặp lại tuple theo số lần đã cho. Cả + và * đều cho kết quả là một tuple mới.

```
# Nối 2 tuple # Output: (2, 4, 6, 3, 5, 7)
print((2, 4, 6) + (3, 5, 7))
# Lặp lại tuple
# Output: ('This is my Python', 'This is my Python', 'This is my Python')
print(("This is my Python",) * 3)
```

6. Xóa tuple

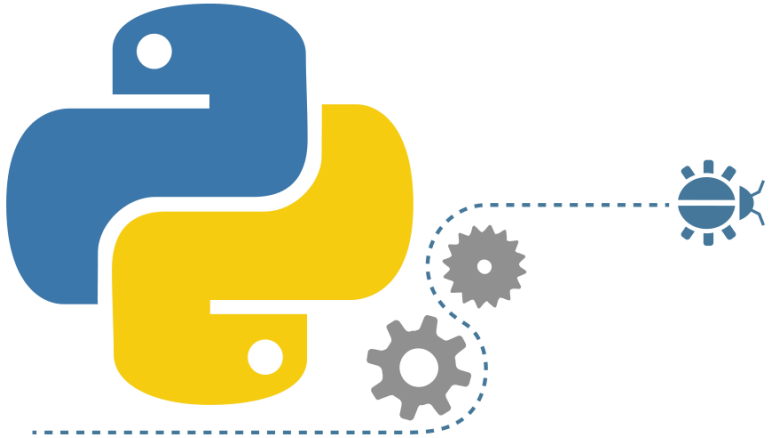


Nội dung bài học

- Các phần tử trong tuple không thể thay đổi nên chúng ta cũng không thể xóa, loại bỏ phần tử khỏi tuple. Nhưng việc xóa hoàn toàn một tuple có thể thực hiện được với từ khóa del như dưới đây:

```
QTM = ['q','u','a','n','t','r','i','m','a','n','g','.','c','o','m']
# TypeError: 'tuple' object doesn't support item deletion
#del QTM[3]
# Có thể xóa toàn bộ tuple
del QTM
QTM
```

7. Phương thức và hàm dùng với tuple trong Python

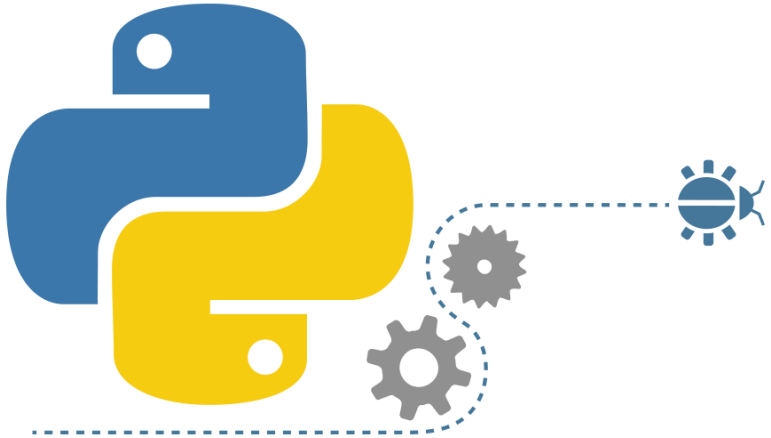


Nội dung bài học

Phương thức thêm phần tử và xóa phần tử không thể sử dụng với tuple, chỉ có 2 phương thức sau là dùng được:

- **count(x)**: Đếm số phần tử x trong tuple.
- **index(x)**: Trả về giá trị index của phần tử x đầu tiên mà nó gặp trong tuple.
- **all()**: Trả về giá trị True nếu tất cả các phần tử của tuple là true hoặc tuple rỗng.
- **any()**: Trả về True nếu bất kỳ phần tử nào của tuple là true, nếu tuple rỗng trả về False.
- **enumerated()**: Trả về đối tượng enumerate (liệt kê), chứa cặp index và giá trị của tất cả phần tử của tuple.
- **len()**: Trả về độ dài (số phần tử) của tuple.
- **max()**: Trả về phần tử lớn nhất của tuple.
- **min()**: Trả về phần tử nhỏ nhất của tuple.
- **sorted()**: Lấy phần tử trong tuple và trả về list mới được sắp xếp (tuple không sắp xếp được).
- **sum()**: Trả về tổng tất cả các phần tử trong tuple.
- **tuple()**: Chuyển đổi những đối tượng có thể lặp (list, string, set, dictionary) thành tuple.

8. Kiểm tra phần tử trong tuple

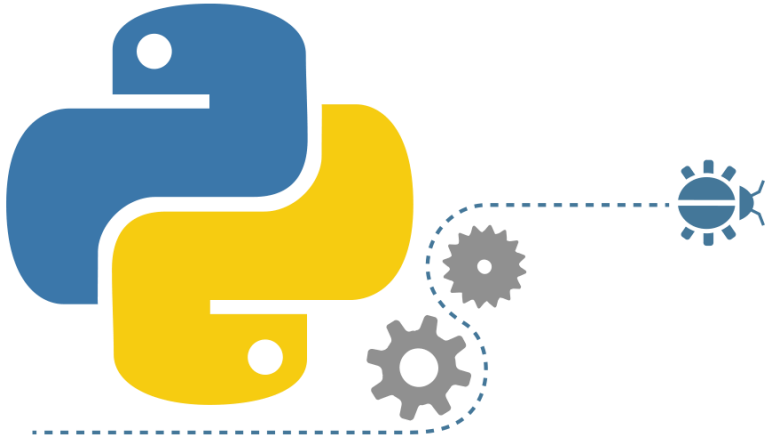


Nội dung bài học

- Bạn có thể kiểm tra xem một phần tử đã tồn tại trong tuple hay chưa với từ khóa in.

```
QTM = ['q','u','a','n','t','r','i','m','a','n','g','.','c','o','m']
# Kiểm tra phần tử
# Output: True
print('a' in QTM)
# Output: False
print('b' in QTM)
# Not in operation
# Output: False
print('g' not in QTM)
```

9. Lặp qua các phần tử của tuple trong Python



Nội dung bài học

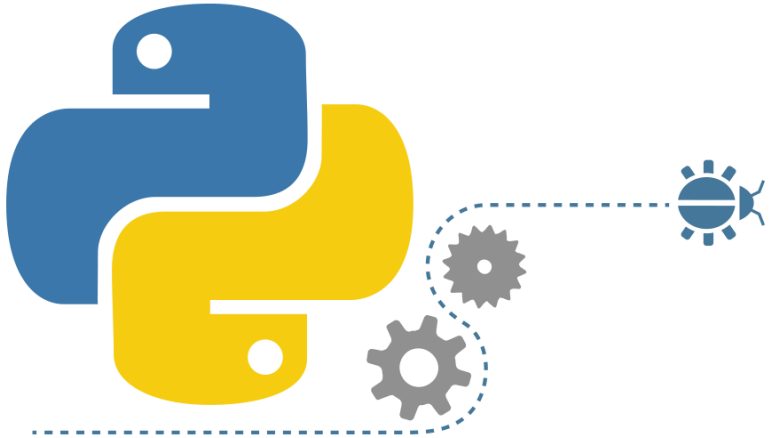
- Sử dụng vòng lặp for để lặp qua các phần tử trong tuple.

```
for ngon_ngu in ('Python','C++','Web'):
print("Tôi thích lập trình",ngon_ngu)
```

- Kết quả trả về sẽ như sau:

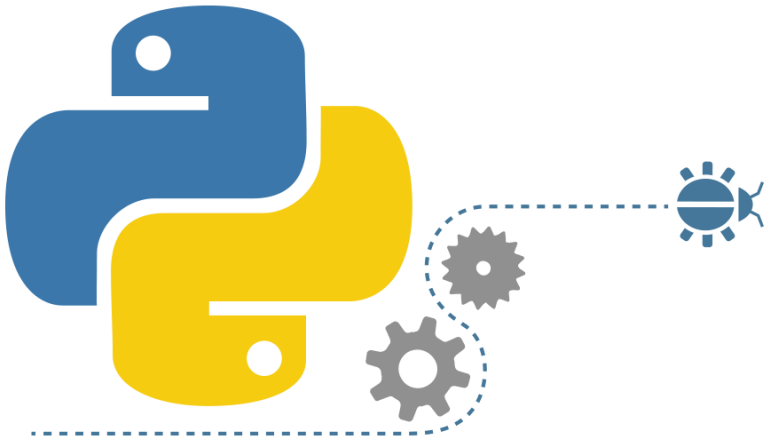
```
Tôi thích lập trình Python
Tôi thích lập trình C++
Tôi thích lập trình Web
```

10. Định nghĩa Dictionary

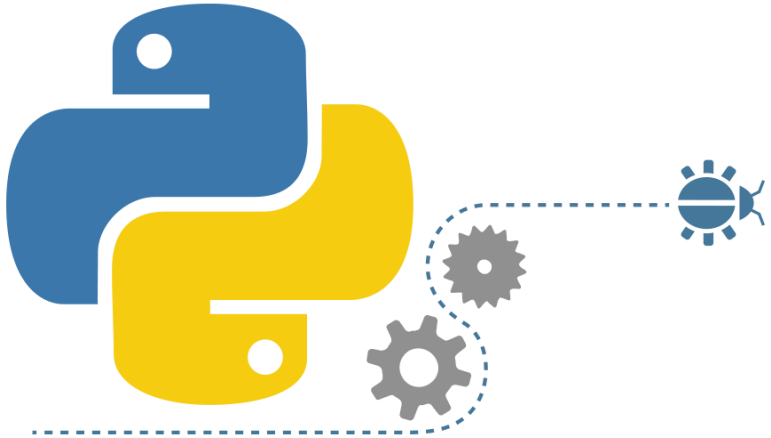


Nội dung bài học

Dictionary là tập hợp các cặp khóa giá trị không có thứ tự. Nó thường được sử dụng khi chúng ta có một số lượng lớn dữ liệu. Các dictionary được tối ưu hóa để trích xuất dữ liệu với điều kiện bạn phải biết được khóa để lấy giá trị.



11. Cách tạo dictionary trong Python



Nội dung bài học

Trong Python, dictionary được định nghĩa trong dấu ngoặc nhọn {} với mỗi phần tử là một cặp theo dạng key:value. Key và value này có thể là bất kỳ kiểu dữ liệu nào. Bạn cũng có thể tạo dictionary bằng cách sử dụng hàm dict() được tích hợp sẵn.

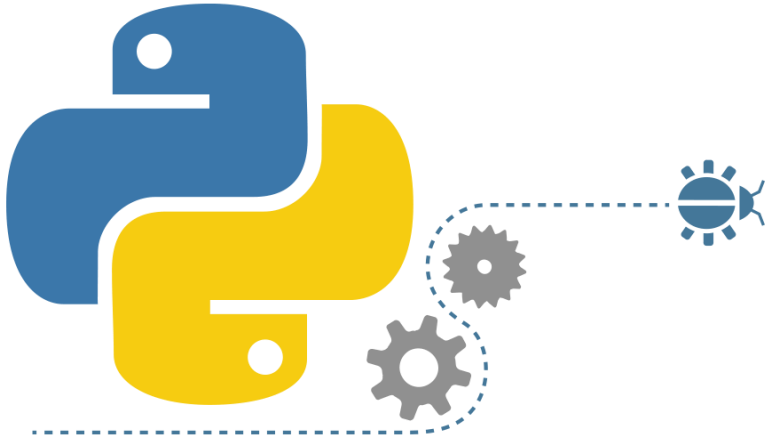
Ví dụ:

```
dict1 = {} #dictionary rỗng

#dict2 là dictionary với các khóa nguyên
dict2 = {1: 'This is my Python',2: 'Công nghệ'}
#Tạo dictionary với khóa hỗn hợp
dict3 = {'tên': 'QTM', 1: [1, 3, 5]}
#Tạo dictionary bằng dict()
dict4 = dict({1:'apple', 2:'ball'})
#Tạo dictionary từ chuỗi với mỗi mục là một cặp
dict5 = dict([(1,'QTM'), (2,'CN')])
```

Khi thực hiện kiểm tra kiểu dữ liệu của d ta được kết quả:

```
>>> type(dict2)
 <class 'dict'>
```

12. Truy cập phần tử của dictionary



Nội dung bài học

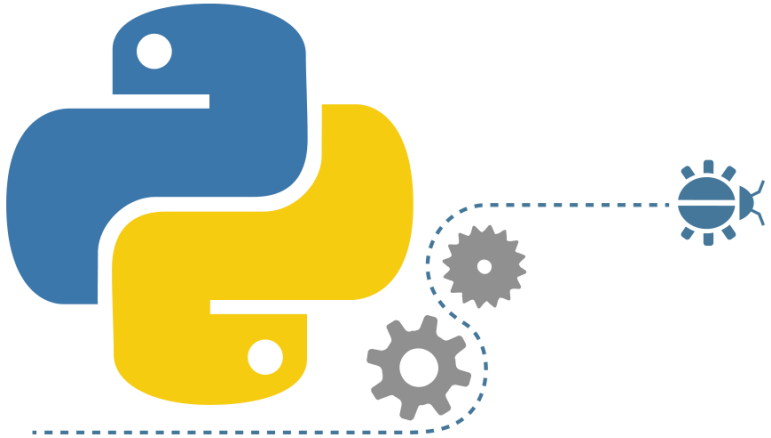
- Các kiểu dữ liệu lưu trữ khác sử dụng index để truy cập vào các giá trị thì dictionary sử dụng các key. Key có thể được sử dụng trong cặp dấu ngoặc vuông hoặc sử dụng get().
- Sử dụng khóa để trích xuất dữ liệu:

```
#khai báo và gán giá trị dict2
 dict2 = {1: 'This is my Python','coding':
'Công nghệ'}
 print(type(dict2)) #in kiểu dữ liệu của
dict2
 #trích xuất dữ liệu bằng khóa rồi in
 print("dict2[1] = ", dict2[1])
 print("dict2[coding] = ",dict2['coding'])
```

Chạy đoạn code trên ta sẽ được kết quả:

```
<class 'dict'>
 dict2[1] = This is my Python
 dict2[coding] = Công nghệ
```

13. Thay đổi, thêm phần tử cho dictionary

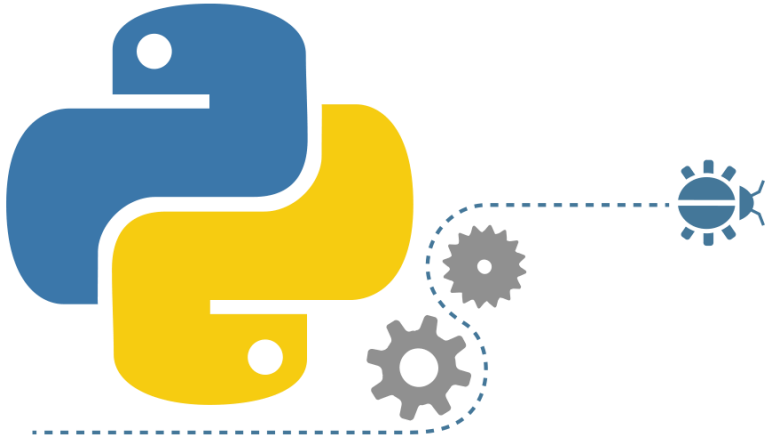


Nội dung bài học

Dictionary có thể thay đổi, nên có thể thêm phần tử mới hoặc thay đổi giá trị của các phần tử hiện có bằng cách sử dụng toán tử gán. Nếu key đã có, giá trị sẽ được cập nhật, nếu là một cặp key: value mới thì sẽ được thêm thành phần tử mới.

```
dict2 = {1: 'This is my Python','coding': 'Công
nghệ'}

 #cập nhật giá trị
 dict2['coding'] = 'This is my Python'

 #output: {1: 'This is my Python', 'codding':
'Công nghệ'}
 print(dict2)

 #thêm phần tử mới
 dict2[2] = 'Python'

 #output: {1: 'This is my Python', 'coding':
'Công nghệ', 2: 'Python'}
 print(dict2)
```

14. Xóa phần tử từ dictionary

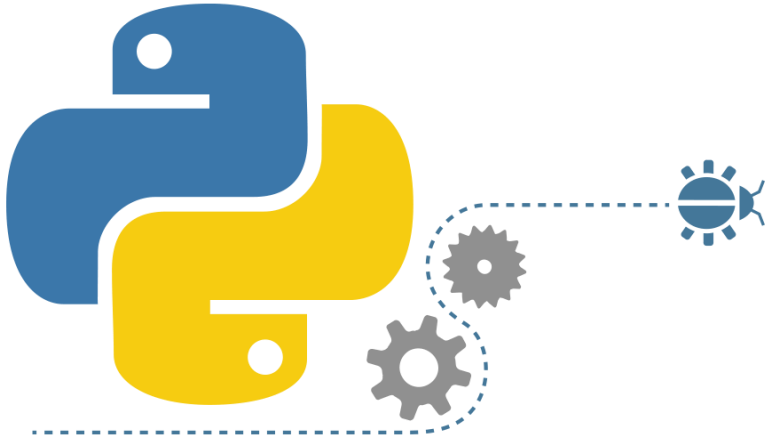


Nội dung bài học

Để xóa phần tử của dictionary

- **pop()**, nó sẽ phần tử có key đã cho và trả về giá trị của phần tử.
- **popitem()** có thể xóa và trả về một phần tử tùy ý dưới dạng (key, value).
- **clear()** tất cả các phần tử trong dictionary sẽ bị xóa cùng lúc.
- **del** cũng có thể dùng để xóa một phần tử hoặc toàn bộ dictionary.

15. Các phương thức và hàm cho dictionary



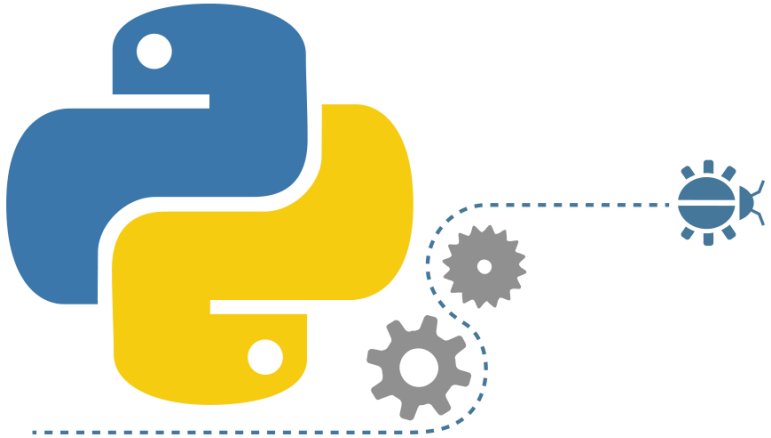
Nội dung bài học

Hàm	Mô tả
clear()	Xóa tất cả phần tử của dictionary.
copy()	Trả về một bản sao shollow copy của dictionary.
fromkeys(seq[,v])	Trả về dictionary mới với key từ seq và value bằng v (default là None).
get(key[,d])	Trả về giá trị của key, nếu key không tồn tại, trả về d. (default là None).
items()	Trả lại kiểu xem mới của các phần tử trong dictionary (key, value).
keys()	Trả về kiểu xem mới của các key trong dictionary.
pop(key[,d])	Xóa phần tử bằng key và trả về giá trị hoặc d nếu key không tìm thấy. Nếu d không được cấp, key không tồn tại thì sẽ tạo lỗi KeyError.
popitem()	Xóa và trả về phần tử bất kỳ ở dạng (key, value). Tạo lỗi KeyError nếu dictionary rỗng.
setdefault(key,[,d])	Nếy key tồn tại trả về value của nó, nếu không thêm key với value là d và trả về d (default là None).
update([other])	Cập nhật dictionary với cặp key/value từ other, ghi đè lên các key đã có.
values()	Trả về kiểu view mới của value trong dictionary.

Các hàm tích hợp như

all(), any(), len(), cmp(), sorted(),... thường được sử dụng với dictionary để thực hiện những nhiệm vụ khác nhau.

16. Dictionary comprehension trong Python



Nội dung bài học

Dictionary comprehension là cách đơn giản, rút gọn để tạo dictionary mới từ một vòng lặp trong Python. Câu lệnh sẽ bao gồm một cặp biểu thức (key:value) cùng câu lệnh for, tất cả đặt trong dấu {}.

```
lap_phuong = {x: x*x*x for x in range(6)}
# Output: {0: 0, 1: 1, 2: 8, 3: 27, 4: 64, 5: 125}
print(lap_phuong)
```

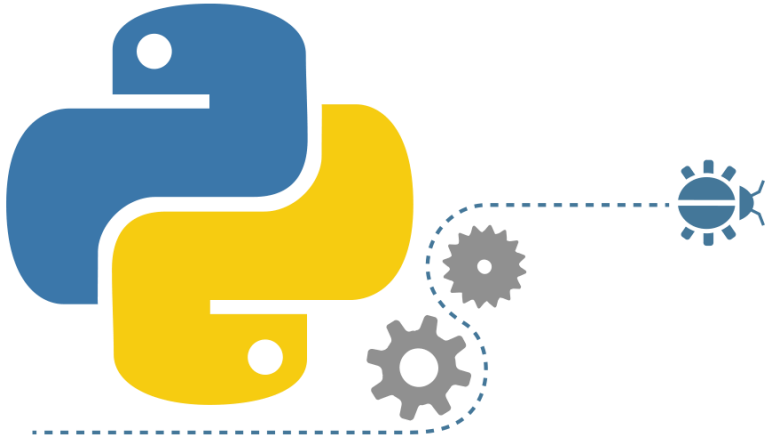
Chương trình trên tương đương với

```
lap_phuong = {}
for x in range(6):
lap_phuong[x] = x*x*x
print(lap_phuong)
```

Bạn cũng có thể sử dụng lệnh if trong dictionary comprehension. Lệnh if có thể lọc những phần tử trong dictionary hiện có để tạo thành dictionary mới như ví dụ dưới đây:

```
lap_phuong_chan = {x: x*x*x for x in range (10)
if x%2==0}
# output: {0: 0, 2: 8, 4: 64, 6: 216, 8: 512}
print(lap_phuong_chan)
```

17. Kiểm tra và lặp qua phần tử trong dictionary



Nội dung bài học

Bạn chỉ có thể kiểm tra key của phần tử đã có trong dictionary hay chưa bằng cách dùng in, và không thể làm điều đó với value.

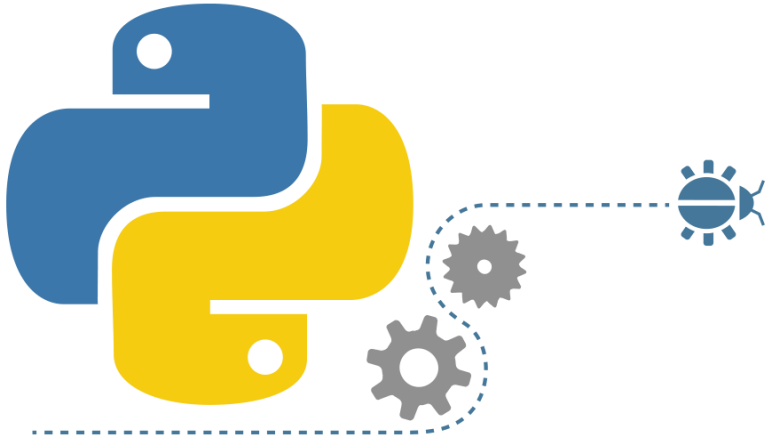
```
lap_phuong = {0: 0, 1: 1, 2: 8, 3: 27, 4: 64, 5: 125}
 #output: True
 print (2 in lap_phuong)
 #output: False
 print (9 in lap_phuong)
 #output: False
 print (5 not in lap_phuong)
```

Bạn có thể sử dụng vòng lặp for để lặp qua key của các phần tử trong dictionary.

```
lap_phuong = {0: 0, 1: 1, 2: 8, 3: 27, 4: 64, 5: 125}
 for i in lap_phuong
 print(lap_phuong[i])
```

Chạy code trên, các giá trị tương ứng với từng key sẽ được in ra màn hình theo thứ tự của key.

18. Bài tập rèn luyện Dictionany



Nội dung bài học

Bài 1:

Câu hỏi:

Viết chương trình và in giá trị theo công thức cho trước: Q = √([(2 * C * D)/H]) (bằng chữ: Q bằng căn bậc hai của [(2 nhân C nhân D) chia H]. Với giá trị cố định của C là 50, H là 30. D là dãy giá trị tùy biến, được nhập vào từ giao diện người dùng, các giá trị của D được phân cách bằng dấu phẩy.

Ví dụ: Giả sử chuỗi giá trị của D nhập vào là 100,150,180 thì đầu ra sẽ là 18,22,24.

Gợi ý:

- Nếu đầu ra nhận được là một số dạng thập phân, bạn cần làm tròn thành giá trị gần nhất, ví dụ 26.0 sẽ được in là 26.
- Trong trường hợp dữ liệu đầu vào được cung cấp cho câu hỏi, nó được giả định là đầu vào do người dùng nhập từ giao diện điều khiển.

Nội dung bài học

Bài 2

Câu hỏi:

Viết một chương trình có 2 chữ số, X, Y nhận giá trị từ đầu vào và tạo ra một mảng 2 chiều. Giá trị phần tử trong hàng thứ i và cột thứ j của mảng phải là i*j.

Lưu ý: i=0,1,...,X-1; j=0,1,...,Y-1.

Ví dụ: Giá trị X, Y nhập vào là 3,5 thì đầu ra là: [[0, 0, 0, 0, 0], [0, 1, 2, 3, 4], [0, 2, 4, 6, 8]]

Gợi ý:

Viết lệnh để nhận giá trị X, Y từ giao diện điều khiển do người dùng nhập vào.